

[P3] Sixteen cards

Mohamed A. Osman — ORCID 0009-0004-5912-999X

Licence: CC BY 4.0. Research draft.

Abstract

No pairwise intersecting 6-uniform family of sixteen cards has transversal number six. A hypothetical counterexample is reduced to one of 463 canonical eight-card cores; for each core the completion problem is written as a CNF formula; all 463 are unsatisfiable; and each unsatisfiability carries a DRAT certificate accepted by an independent checker. With [P2] this gives $g(6) \geq 17$.

This is the computer-assisted stage of the project. The reduction and the encoding are proof obligations that live in the source, not in the certificates.

Computer-assisted; not independently reviewed. The reduction and the encoding are open obligations no certificate discharges. No priority claimed. See the note at the end.

1. Setting

Let F be a pairwise intersecting 6-uniform family of sixteen cards with no five-cover.

By [P2, Cor 2] the maximum symbol degree is exactly four.

Fix a symbol x of degree four and delete its four cards. Twelve cards remain; call them the **kernel** G .

Proposition 1. $\tau(G) = 5$.

Proof. If $\tau(G) \leq 4$ then x together with a four-cover of G is a five-cover of F . And deleting x from any of the four removed cards leaves five symbols meeting every card of G , since each card of G meets that card and cannot do so through x . $\square$

So the four deleted cards are $\{x\} \cup C_i$ for $i = 1, \dots, 4$, where each C_i is a five-element transversal of G .

Proposition 2 (extension criterion). *Four five-covers $C_1, \dots, C_4$ of G produce a sixteen-card family with $\tau = 6$ if and only if every five-cover T of G is disjoint from at least one C_i .*

Proof. If some T met all four, then T would cover the twelve kernel cards and also the four new cards, a five-cover of F . Conversely, a five-cover of F containing x would cover G with at most four other symbols, contradicting Proposition 1; and a five-cover not containing x is a five-cover T of G , which by hypothesis misses one new card. Any single card is a six-cover, so $\tau(F) = 6$ exactly. $\square$

In the language of graphs: form the **disjointness graph** on the five-covers of G , joining two when they are disjoint. Then $C_1, \dots, C_4$ must be a *total dominating set* — every vertex, including a selected one, needs a selected neighbour. This is not ordinary domination, where a selected vertex may dominate itself.

Degree capacities. For a symbol z of G , let $h(z)$ count its occurrences among the four selected covers. Since the final degree cannot exceed four,

$$d_G(z) + h(z) \leq 4.$$

So a degree-four symbol of G cannot be used in any new card, a degree-three symbol occurs in at most one C_i , and a degree-two symbol in at most two. In particular every symbol of $C_i \cap C_j$ has kernel degree at most two.

2. The second deletion

The kernel G may itself contain a symbol of degree four. Both cases are handled by the same object.

If y is a degree-four symbol of G , its four cards are disjoint from the four cards of x , and deleting both leaves eight cards H .

Proposition 3. $\tau(H) = 4$ and the maximum degree in H is at most three.

Proof. $\tau(H) \geq 4$: a three-cover of H together with x and y would be a five-cover of F . $\tau(H) \leq 4$ is [P2, Thm 1] applied after padding, or directly. If some symbol had degree four in H , deleting its cards would leave at most four, coverable by two symbols, so that symbol plus two plus x plus y is a five-cover. $\square$

Proposition 4. *The degree-three symbols of H , read as triples of cards, are pairwise intersecting.*

Proof. Two disjoint triples cover six cards with two symbols; the remaining two cards share a symbol; together with x and y that is a five-cover. $\square$

An eight-card core with these properties exists — see [P2, §5] for an explicit one — so this stage cannot be closed by a lemma the way eleven cards were. The cores must be enumerated.

3. The 463 canonical cores

The enumeration runs in four steps, each with its own script and output.

Step 1. Maximal intersecting triple systems on eight points. Enumerate the maximal cliques of the graph whose vertices are the 56 triples from eight points and whose edges join intersecting triples, keeping those whose union is all eight points. There are **10,144**, of sizes $\{21:8, 16:336, 14:1680, 12:8120\}$. Script: `code/enumerate_eight_maximal.py`.

Step 2. Orbits. Under the symmetric group on the eight points these fall into **seven** orbits, of sizes

$$8, \quad 280, \quad 1680, \quad 56, \quad 560, \quad 2520, \quad 5040,$$

summing to 10,144. Script: `code/verify_eight_orbits.py`, which rebuilds each orbit from a representative and checks it equals the observed class.

Step 3. Triple systems. For each of the seven representatives, enumerate the ways of assigning to every card a triple of the maximal family containing it. That gives **39,768** triple systems in total, distributed over the seven representatives as 28329, 4932, 1526, 3858, 156, 535, 432. Reducing by the stabiliser leaves **129** symmetry classes, whose multiplicities sum back to 39,768. Scripts: `code/enumerate_eight_triples.py`, `code/classify_eight_triples.py`.

Step 4. Completions. Each class is completed with the degree-two symbols forced by the uncovered pairs, then with the remaining slack filled by binary and singleton symbols. That gives **772** raw completions, which reduce to **463** canonical cores. Script: `code/enumerate_eight_cores.py`; output `data/eight_cores.jsonl`.

Each line of that file records **supports**, a list of symbol supports as subsets of $\{0, \dots, 7\}$, and a **multiplicity**. `code/verify_eight_orbits.py` also checks that every one of the 463 cores really is eight distinct six-element cards, pairwise intersecting, with no three symbols covering all eight.

Lemma 1 (finite reduction, computer-assisted). *Every hypothetical sixteen-card counterexample is represented by one of the 463 cores in `data/eight_cores.jsonl`.*

The completeness of this reduction is a proof obligation on the four scripts above. It is not certified by anything downstream.

4. The CNF encoding

For a fixed core, the remaining unknown is how the eight new cards — the four containing x and the four containing y — are filled.

Positions $0, \dots, 7$ index the eight new cards, the first four being the x -block and the last four the y -block. The variables are:

- $h_{v,j}$ for each core symbol v and each new card j : the core symbol v also appears in new card j ;
- z_b for each subset $b \subseteq \{0, \dots, 7\}$ of size 2, 3 or 4 meeting both blocks: a fresh symbol whose support is exactly b .

The clauses enforce:

1. **degree caps** — core symbol v with existing support s may take at most $4 - |s|$ new cards, and a singleton must take all of them;
2. **card size** — for each new card j , exactly five of the core symbols and fresh symbols lie in it, the sixth being x or y ;
3. **intersection with the core** — each new card meets every core card;
4. **cross-block intersection** — each x -card and each y -card share a core symbol or a fresh crossing symbol;
5. **antichain and support constraints** of the canonical representation, so that no fresh symbol's support contains another's and no core symbol's completed support is contained in another's;
6. **no five-cover** — for every four-element and every five-element subset of the symbol set that already covers the eight core cards, clauses forbidding it from also meeting all eight new cards;
7. **symmetry breaking** — lexicographic constraints along the chain $(0, 1), (1, 2), (2, 3), (4, 5), (5, 6), (6, 7), (0, 4)$, which are legitimate because the four x -cards are interchangeable, the four y -cards are interchangeable, and x and y may be swapped.

Implementation: `code/search_sixteen.py`, function `build`. The same function exports the DIMACS files through `code/export_core_cnf.py`, so the CNF a reviewer checks is the one the search used.

The encoding is part of the mathematics. A DRAT certificate says the formula has no model; it says nothing about whether the formula means what §1 to §3 mean. That reading has to be done in the source.

5. Results

All 463 instances are unsatisfiable.

The first pass used a 50,000-conflict budget and returned 460 UNSAT and three UNKNOWN — cores 7, 239 and 342. Budget exhaustion is never treated as exclusion: those three were rerun without a budget and proved UNSAT. `code/resolve_sixteen.py` carries that rule explicitly.

core	variables	clauses	conflicts	solve time (s)
7	40,583	210,333	52,765	7.406
239	38,280	199,136	69,317	9.594
342	36,922	190,406	83,403	11.453

Raw records are in `results/sat_results_cadical195_0.jsonl` and the retry files.

6. DRAT certification

For each core the CNF was written in DIMACS form, a standalone CaDiCaL produced a DRAT proof, and `drat-trim` checked that proof independently. The batch finished with

```
VERIFIED: 462
SKIPPED  : 1
FAILED   : 0
```

The skipped instance was core 0, which had already been verified separately before the batch, so the combined total is 463 of 463 verified with none failing. Per-core records are in `results/certificate_summary.jsonl`; the three heaviest are

core	DRAT bytes	checker	time (s)
7	10,110,384	VERIFIED	6.658
239	5,858,286	VERIFIED	2.792
342	7,661,933	VERIFIED	7.252

The CNF and DRAT files for those three are shipped in `data/cnf/` so that the certification step can be reproduced without a solver. The full archive of 463 pairs is large and belongs in a release asset or an archive record rather than in the repository.

7. The trusted boundary

A checked DRAT proof establishes that the generated formula is unsatisfiable. It removes the SAT solver from the trusted base. It does **not** establish any of:

- that every sixteen-card counterexample reaches one of the 463 cores;
- that `data/eight_cores.jsonl` is complete and free of accidental omission;
- that each CNF variable and clause has the intended combinatorial meaning.

Those three are the reduction and encoding obligations. They are audited through the source, the canonical data, the structural counts of §3, and the controls recorded in `AUDIT.md`. With them accepted, the 463 checked contradictions leave no counterexample.

Theorem 1 (computer-assisted). *There is no pairwise intersecting 6-uniform family of sixteen cards with transversal number six. With [P2, Cor 1], $g(6) \geq 17$.*

8. Verification

claim	script or data
10,144 maximal systems, sizes	<code>code/enumerate_eight_maximal.py</code>
seven orbits, sizes as listed	<code>code/verify_eight_orbits.py</code>
39,768 triple systems, 129 classes	<code>code/classify_eight_triples.py</code> , <code>data/eight_triple_classes.jsonl</code>
772 raw, 463 canonical cores	<code>code/enumerate_eight_cores.py</code>
every core is a genuine eight-card family with $\tau = 4$	<code>code/verify_eight_orbits.py</code>
the CNF is the one <code>build</code> produces	<code>code/verify_cnf_regeneration.py</code>
the encoding is not trivially over-constrained	<code>code/verify_encoding_controls.py</code>
463/463 UNSAT	<code>results/sat_results_cadical195_0.jsonl</code> , <code>results/sat_retry*.jsonl</code>
463/463 DRAT-verified	<code>results/certificate_summary.jsonl</code> , <code>data/cnf/</code>

This work was prepared with AI assistance (ChatGPT, OpenAI; Claude, Anthropic), used for algebraic derivation, for drafting and rewriting code and text, for running and re-running the computations, and for auditing the papers against their own scripts. The same tools were used in the review, so the review carries the same caveat. All statements were checked by the author, who is responsible for them. No priority is claimed and the result has not been independently reviewed by a human or a proof assistant; the repository README sets out the division of labour and the open obligations in full.

References

1. J. Barát, *Intersecting and 2-intersecting hypergraphs with maximal covering number: the Erdős–Lovász theme revisited*, arXiv:2011.04444.
2. V. Sivashankar, *An Improved Lower Bound for the Erdős–Lovász Cover Number Problem*, arXiv:2606.24878v2.
3. M. J. H. Heule, W. A. Hunt Jr., N. Wetzler, *Trimming while checking clausal proofs*, FMCAD 2013. The `drat-trim` checker.